UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

Progetto per:

Basi di Dati 2025/2026

Prof. Vincenzo Moscato

Realizzato da:

Giovanni Liguori N46006945

Indice

1. **Introduzione** ……. 3

2. **Creazione DB** ……. 3

2.1 Specifiche di progetto ……. 3

2.2 Progettazione concettuale ……. 5

2.3 Progettazione logica ……. 10

2.4 Progettazione fisica ……. 15

3. **Ottimizzazione DB** ……. 27

3.1 Indici ……. 27

3.2 Gestione della concorrenza ……. 28

3.3 Gestione dell'affidabilità ……. 29

3.4 Gestione della recovery ……. 30

3.5 Gestione della sicurezza ……. 31

4. **Funzionalità DB** ……. 32

4.1 Trigger ……. 32

4.2 Stored procedure ……. 35

4.3 Viste ……. 37

4.4 Query ……. 38

5. **Oracle APEX** ……. 40

5.1 Pagine realizzate ……. 40

5.2 Query in linguaggio naturale ……. 44

1. Introduzione

Il progetto Click Advisor nasce con l'obiettivo di raccogliere le opinioni di milioni di utenti sulle attività commerciali che frequentano ogni giorno. Ogni utente può dire la sua opinione, dare suggerimenti ai business e pubblicarne fotografie. Allo stesso modo business di ogni categoria possono presentarsi al web mostrando agli utenti i propri orari di apertura e le caratteristiche che li rendono appetibili. Per rendere ciò possibile Click Advisor propone un'architettura sicura e resiliente, per rendere a utenti e business la migliore esperienza possibile.

2. Creazione DB

Per creare il nostro database dobbiamo passare per diverse basi progettuali: iniziamo subito con la raccolta delle specifiche di progetto, continuando con la progettazione concettuale e terminando con la progettazione logica. Solo una volta completate queste fasi potremmo poi procedere con la realizzazione fisica del database Oracle.

2.1 Specifiche di progetto

Le Specifiche di progetto sono il ponte tra il linguaggio naturale (spesso confuso o incompleto) di un cliente e la struttura tecnica di un database. In pratica, sono l'insieme di regole, dati e processi che il sistema dovrà gestire.

Vediamo le specifiche che abbiamo raccolto dal nostro caso reale.

Specifiche delle attività:

- identificata da un *codice univoco* e caratterizzata da un insieme di informazioni descrittive: *nome, indirizzo, città, stato, CAP,* coordinate geografiche (*latitudine e longitudine*), *valutazione media in stelle*, *numero di recensioni totali* e *stato* (aperto o chiuso);

- può appartenere a una o più *categorie*;
- possiede un insieme di *attributi*;
- devono essere memorizzati gli *orari di apertura*, distinti per giorno della settimana;

Specifiche degli utenti:

- è caratterizzato da *nome*, *data di iscrizione*, *numero di recensioni scritte*, *lista di amici* (altri utenti), *numero di complimenti ricevuti* (suddivisi per tipo: "cool", "funny", "useful", ecc.) e *valutazione media delle recensioni pubblicate*;
- può scrivere *recensioni*;
- può pubblicare *suggerimenti rapidi*;
- può registrare i *check-in* presso le attività;
- può pubblicare *foto* di attività;

Specifiche delle recensioni:

- identificata da un *codice univoco* e contiene *data*, *valutazione* (in stelle), *testo* e *numero di voti* ricevuti dagli altri utenti (utili, divertenti, interessanti);
- è associata a un *utente* e ad *un'attività*;

Specifiche dei suggerimenti rapidi:

- contiene un *breve testo*, la *data di pubblicazione* e il *numero di complimenti ricevuti*;
- è associata a un *utente* e ad *un'attività*;

Specifica dei check-in:

- associato a un *utente* ed *un'attività*;

- associato ad una o più *date* e *orari* di visita;

Specifiche sulle fotografie

- identificata da un *codice* e caratterizzata da una *didascalia* e da un'*etichetta*;
- caricata da un *utente* e associata ad un'*attività*.

A queste specifiche riferite che ci guidano alla progettazione concettuale del database si aggiungono anche specifiche di sicurezza che definiscono come i singoli utilizzatore finali possono interfacciarsi al prodotto finale.

Specifiche di sicurezza:

- distinguere i diversi ruoli applicativi:
    - ***Admin***: disporranno di privilegi completi di gestione sull'intero sistema e potranno operare su tutti i dati, comprese le attività commerciali, le recensioni e i profili utente;
    - ***Moderator***: potranno approvare, modificare o eliminare contenuti generati dagli altri utenti;
    - ***User***: creare, modificare o eliminare esclusivamente i propri contenuti (recensioni, suggerimenti, fotografie e check-in) e interagire con gli altri utenti attraverso voti, complimenti e relazioni di amicizia;

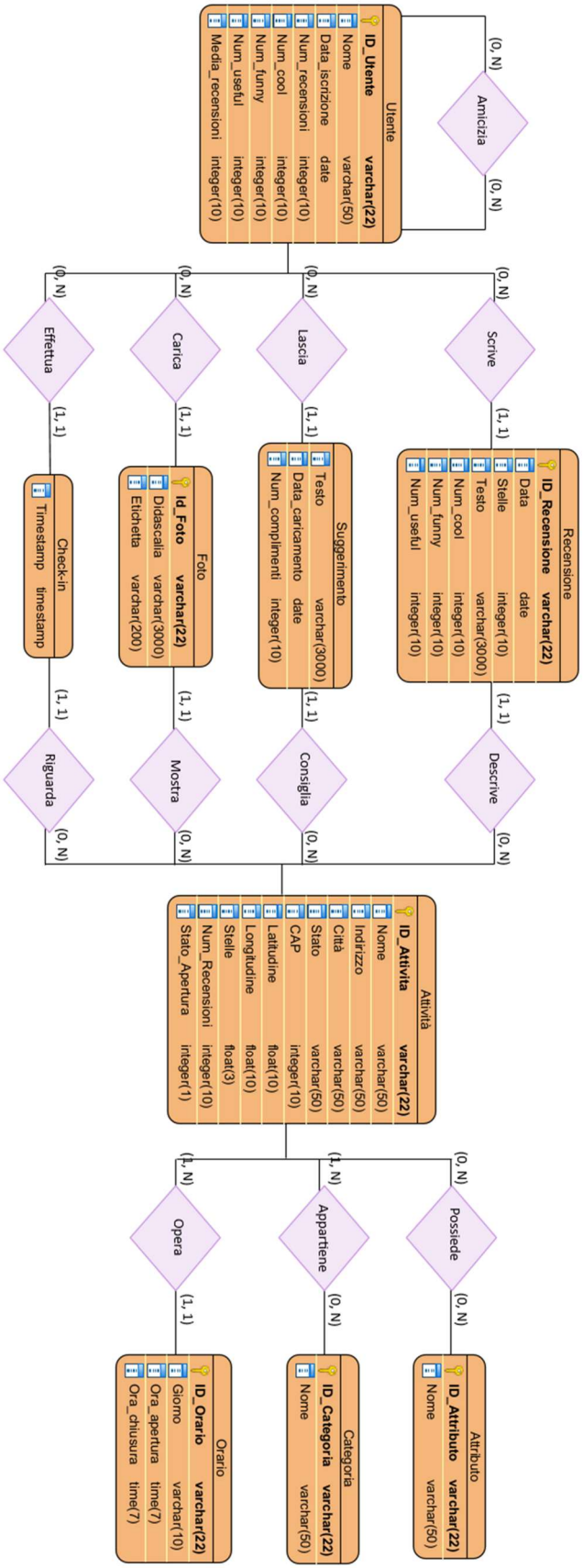
2.2 Progettazione concettuale

La progettazione concettuale è la fase più creativa e astratta della creazione di un database. Il suo obiettivo è trasformare le specifiche raccolte (spesso in linguaggio naturale) in una rappresentazione formale, completa e indipendente dal software che verrà usato nella realizzazione fisica.

Il prodotto finale della fase di progettazione concettuale è il ***modello Entità-Relazione*** (ER); questo modello ci permette di rappresentare la nostra realtà di interesse attraverso tre componenti principali:

- ***Entità***: un'entità è una collezione di occorrenze di entità, fisiche o astratte, che possiedono le stesse proprietà;
- ***Relazioni*** (o Associazioni): una relazione è un legame concettuale fra due o più entità. Le relazioni sono inoltre dotate di cardinalità, ovvero una coppia di valori associati ad ogni entità che specificano il numero minimo e massimo delle occorrenze di associazioni cui ciascuna occorrenza di entità può partecipare;
- ***Attributi***: gli attributi sono le proprietà descrittive di una data entità. Tra questi figurano anche gli identificatori univoci, ovvero uno o più attributi che distinguono univocamente ciascuna occorrenza di entità;

Nella pagina successiva, mostriamo il diagramma ER della nostra realtà di interesse, realizzato tramite Visual Paradigm Community Edition:


Utente
ID_Utente varchar(22)
Nome varchar(50)
Data_iscrizione date
Num_recensioni integer(10)
Num_cool integer(10)
Num_funny integer(10)
Num_useful integer(10)
Media_recensioni integer(10)
Amicizia
(0, N)
(0, N)
Recensione
ID_Recensione varchar(22)
Data date
Stelle integer(10)
Testo varchar(3000)
Num_cool integer(10)
Num_funny integer(10)
Num_useful integer(10)
Scrive
Descrive
(0, N)
(1, 1)
(1, 1)
(0, N)
Lascia
Carica
Effettua
Suggerimento
Testo varchar(3000)
Data_caricamento date
Num_complimenti integer(10)
Foto
Id_Foto varchar(22)
Didascalia varchar(3000)
Etichetta varchar(200)
Check-in
Timestamp timestamp
Consiglia
Mostra
Riguarda
Attività
ID_Attività varchar(22)
Nome varchar(50)
Indirizzo varchar(50)
Città varchar(50)
Stato varchar(50)
CAP integer(10)
Latitudine float(10)
Longitudine float(10)
Stelle float(3)
Num_Recensioni integer(10)
Stato_Apertura integer(1)
Possiede
Appartiene
Opera
Attributo
ID_Attributo varchar(22)
Nome varchar(50)
Categoria
ID_Categoria varchar(22)
Nome varchar(50)
Orario
ID_Orario varchar(22)
Giorno varchar(10)
Ora_apertura time(7)
Ora_chiusura time(7)

A partire dalla realtà di riferimento in esame è possibile ricavare le seguenti entità dotate degli attributi elencati:

- **Utente**: rappresenta le informazioni relative agli utenti iscritti a ClickAdvisor; gli attributi sono: *ID_Utente, Nome, Data_Iscrizione, Num_recensioni, Num_cool, Num_funny, Num_useful* e *Media_recensioni*;
- **Recensione**: rappresenta le informazioni relative alle recensioni scritte dagli utenti rispetto alle attività; gli attributi sono: *ID_Recensione, Data, Stelle, Testo, Num_cool, Num_funny* e *Num_useful*;
- **Suggerimento**: rappresenta le informazioni relative ai suggerimenti lasciati dagli utenti alle attività che visitano; gli attributi sono: *Testo, Data_caricamento* e *Num_complimenti*;
- **Foto**: rappresenta le informazioni releative alle fotografie che gli utenti caricano per mostrare i business; gli attributi sono: *ID_Foto, Didascalia* ed *Etichetta*;
- **Check-in**: rappresenta le informazioni relative ai check-in che gli utenti effettuano quando visitano un'attività; gli attributi sono: *Timestamp*;
- **Attività**: rappresenta le informazioni relative alle attività registrate a ClickAdvisor; gli attributi sono: *ID_Attivita, Nome, Indirizzo, Città, Stato, CAP, Latitudine, Longitudine, Stelle, Num_recensioni* e *Stato_apertura*;
- **Attributo**: rappresenta le informazioni relative agli attributi che le attività possono possedere; gli attributi sono: *ID_Attributo* e *Nome*;
- **Categoria**: rappresenta le informazioni relative alle categorie a cui le attività possono far parte; gli attributi sono: *ID_Categoria* e *Nome*;

- **Orari**: rappresenta le informazioni relative agli orari in cui le attività sono aperte; gli attributi sono: *ID_Orario, Giorno, Ora_apertura* e *Ora_chiusura*.

Inoltre, sono state ricavate le seguenti relazioni che legano tra di loro gli attributi individuati:

- Tra l'entità Utente e l'entità Utente esiste l'associazione ricorsiva **AMICIZIA**, caratterizzata dal fatto che ciascun utente può possedere uno o più amici (*0, N*);
- Tra l'entità Utente e l'entità Recensione esiste l'associazione **SCRIVE**, caratterizzata dal fatto che un utente può scrivere nessuna o molte recensioni (*0, N*) e una recensione è scritta da uno e un solo utente (*1, 1*);
- Tra l'entità Utente e l'entità Suggerimento esiste l'associazione **LASCIA**, caratterizzata dal fatto che un utente può lasciare nessun o molti suggerimenti (*0, N*) e un suggerimento è scritto da uno e un solo utente (*1, 1*);
- Tra l'entità Utente e l'entità Foto esiste l'associazione **CARICA**, caratterizzata dal fatto che un utente può caricare nessuna o molte foto (*0, N*) e una foto è caricata da uno e un solo utente (*1, 1*);
- Tra l'entità Utente e l'entità Check-in esiste l'associazione **EFFETTUA**, caratterizzata dal fatto che un utente può effettuare nessuno o molti check-in (*0, N*) e un check-in è effettuato da uno e un solo utente (*1, 1*);
- Tra l'entità Recensione e l'entità Attività esiste l'associazione **DESCRIVE**, caratterizzata dal fatto che una recensione descrive una ed una sola attività (*1, 1*) e un'attività può essere descritta da nessuna o molte recensioni (*0, N*);

- Tra l'entità Suggerimento e l'entità Attività esiste l'associazione **CONSIGLIA**, caratterizzata dal fatto che un suggerimento consiglia una ed una sola attività (*1, 1*) e un'attività può essere consigliata da nessun o molti suggerimenti (*0, N*);
- Tra l'entità Foto e l'entità Attività esiste l'associazione **MOSTRA**, caratterizzata dal fatto che una fotografia mostra una ed una sola attività (*1, 1*) e un'attività può essere mostrata da nessuna o molte fotografie (*0, N*);
- Tra l'entità Check-in e l'entità Attività esiste l'associazione **RIGUARDA**, caratterizzata dal fatto che un check-in riguarda una ed una sola attività (*1, 1*) e un'attività può essere riguardata da nessuno o molti check-in (*0, N*);
- Tra l'entità Attività e l'entità Attributo esiste l'associazione **POSSIEDE**, caratterizzata dal fatto che un'attività può possiede nessun o molti attributi (*0, N*) e un attributo può essere posseduto da nessuna o molte attività (*0, N*);
- Tra l'entità Attività e l'entità Categoria esiste l'associazione **APPARTIENE**, caratterizzata dal fatto che un'attività deve appartenere ad una o molte categorie (*1, N*) e una categoria può essere associata a nessuna o molte attività (*0, N*);
- Tra l'entità Attività e l'entità Orario esiste l'associazione **OPERA**, caratterizzata dal fatto che un'attività deve operare in uno o molti orari (*1, N*) e un orario deve essere associato ad una sola attività (*1, 1*);

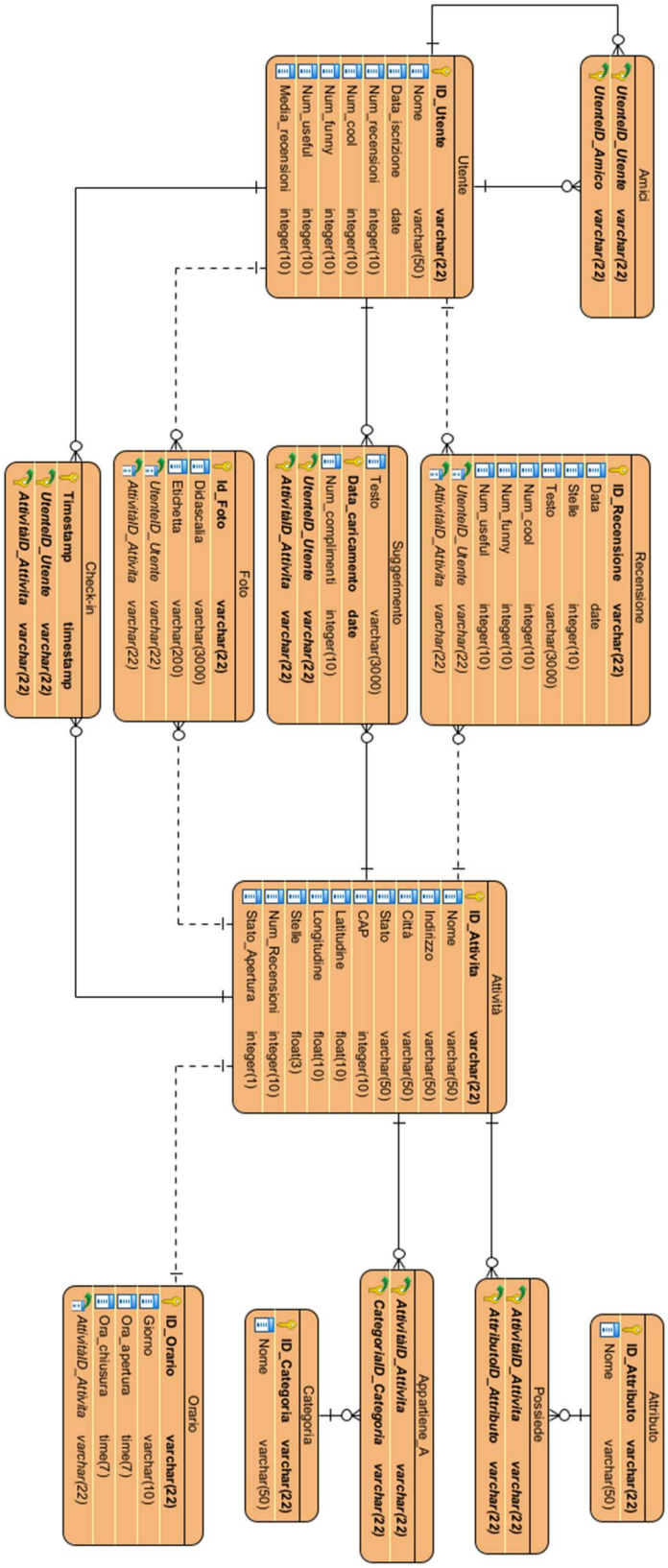
2.3 Progettazione logica

Se la progettazione concettuale è il "disegno" astratto (il cosa), la progettazione logica è la traduzione di quel disegno in una struttura che un computer può effettivamente gestire (il come).

Per poter passare dal modello E/R al ***modello relazionale*** bisogna adottare le opportune regole di traduzione. Verranno ora elencate le regole utilizzate per trasformare le entità e relazioni rilevate precedentemente in tabelle relazionali con le opportune chiavi primarie, chiavi esterne e vincoli di integrità:

- Traduzione delle associazioni **uno a molti**: un'associazione uno a molti dello schema concettuale si traduce aggiungendo alla relazione che traduce l'entità dal lato uno gli attributi dell'associazione e l'identificatore dell'entità dal lato molti, eventualmente ridenominato. Questa regola è stata utilizzata sulle associazioni: *Scrive, Lascia, Carica, Effettua, Descrive, Consiglia, Mostra, Riguarda, Opera*;
- Traduzione delle associazioni **molti a molti**: un'associazione molti a molti dello schema concettuale si traduce in una relazione avente per attributi gli attributi dell'associazione e gli identificatori delle entità coinvolte; l'insieme di tali identificatori costituisce la chiave primaria della relazione. Questa regola è stata utilizzata per tradurre le associazioni: *Possiede, Appartiene*;
- Traduzione delle associazioni **ricorsive**: un'associazione ricorsiva si traduce utilizzando le stesse regole di traduzione delle associazioni classiche; nel nostro caso essendo la relazione *Amicizia* molti a molti è stata usata la regola precedentemente enunciata.

Nella pagina successiva, mostriamo il diagramma relazionale della nostra realtà di interesse, realizzato tramite Visual Paradigm Community Edition:


Amici
UtenteID_Utente varchar(22)
UtenteID_Amico varchar(22)
Utente
ID_Utente varchar(22)
Nome varchar(50)
Data_iscrizione date
Num_recensioni integer(10)
Num_cool integer(10)
Num_funny integer(10)
Num_useful integer(10)
Media_recensioni integer(10)
Recensione
ID_Recensione varchar(22)
Data date
Stelle integer(10)
Testo varchar(3000)
Num_cool integer(10)
Num_funny integer(10)
Num_useful integer(10)
UtenteID_Utente varchar(22)
AttivitaID_Attivita varchar(22)
Suggerimento
Testo varchar(3000)
Data_caricamento date
Num_complimenti integer(10)
UtenteID_Utente varchar(22)
AttivitaID_Attivita varchar(22)
Foto
Id_Foto varchar(22)
Didascalia varchar(3000)
Etichetta varchar(200)
UtenteID_Utente varchar(22)
AttivitaID_Attivita varchar(22)
Check-in
Timestamp timestamp
UtenteID_Utente varchar(22)
AttivitaID_Attivita varchar(22)
Attività
ID_Attivita varchar(22)
Nome varchar(50)
Indirizzo varchar(50)
Città varchar(50)
Stato varchar(50)
CAP integer(10)
Latitudine float(10)
Longitudine float(10)
Stelle float(3)
Num_Recensioni integer(10)
Stato_Apertura integer(1)
Orario
ID_Orario varchar(22)
Giorno varchar(10)
Ora_apertura time(7)
Ora_chiusura time(7)
AttivitaID_Attivita varchar(22)
Appartiene_A
AttivitaID_Attivita varchar(22)
CategoriaID_Categoria varchar(22)
Categoria
ID_Categoria varchar(22)
Nome varchar(50)
Possiede
AttivitaID_Attivita varchar(22)
AttributoID_Attributo varchar(22)
Attributo
ID_Attributo varchar(22)
Nome varchar(50)

Di seguito riportiamo lo ***schema logico relazionale***, ricavato dal diagramma relazionale appena mostrato, con le relative chiavi primarie:

- **UTENTE (ID_Utente**, Nome, Data_Iscrizione, Num_recensioni, Num_cool, Num_funny, Num_useful, Media_recensioni);
- **AMICI** (***UTENTE:ID_Utente, UTENTE:ID_Amico***);
- **RECENSIONE (ID_Recensione**, Data, Stelle, Testo, Num_cool, Num_funny, Num_useful, *UTENTE:ID_Utente, ATTIVITA:ID_Attivita*);
- **SUGGERIMENTO** (Testo, **Data_caricamento**, Num_complimenti, ***UTENTE:ID_Utente, ATTIVITA:ID_Attivita***);
- **FOTO (ID_Foto**, Didascalia, Etichetta, *UTENTE:ID_Utente, ATTIVITA:ID_Attivita*);
- **CHECK-IN (Timestamp, *UTENTE:ID_Utente, ATTIVITA:ID_Attivita***);
- **ATTIVITA (ID_Attivita**, Nome, Indirizzo, Città, Stato, CAP, Latitudine, Longitudine, Stelle, Num_recensioni, Stato_apertura);
- **ATTRIBUTO (ID_Attributo**, Nome);
- **POSSIEDE** (***ATTIVITA:ID_Attivita, ATTRIBUTO:ID_Attributo***);
- **CATEGORIA (ID_Categoria**, Nome);
- **APPARTIENE_A** (***ATTIVITA:ID_Attivita, CATEGORIA:ID_Categoria***);
- **ORARIO (ID_Orario**, Giorno, Ora_apertura, Ora_chiusura, *ATTIVITA:ID_Attivita*).

Lo schema logico relazionale appena presentato rispetta inoltre tutte e tre le **forme normali** eliminando ridondanze e anomalie di aggiornamento:

- **Prima forma normale** (**1NF**): un database è in 1NF se i suoi attributi sono atomici (indivisibili) e se esiste una chiave primaria univoca per ogni tabella:
    - *Atomicità*: ogni colonna contiene un singolo valore. Ad esempio, anziché avere una lista di categorie separate da virgole all'interno della tabella "ATTIVITA", il sistema utilizza la tabella di associazione "APPARTIENE_A" per gestire la relazione molti-a-molti tra attività e categorie;
    - *Chiavi Primarie*: ogni tabella ha una chiave primaria chiaramente definita (es. "PK_UTENTE", "PK_ATTIVITA") che identifica univocamente ogni record;
- **Seconda Forma Normale** (**2NF**): Un database è in 2NF se è in 1NF e se ogni attributo non-chiave dipende interamente dalla chiave primaria (assenza di dipendenze parziali).
    - *Chiavi Singole*: Per tabelle come "UTENTE", "ATTIVITA" e "RECENSIONE", la 2NF è soddisfatta automaticamente poiché la chiave primaria è composta da un solo attributo; non possono quindi esistere dipendenze da una "parte" della chiave
    - *Chiavi Composte*: Nelle tabelle con chiavi composte, come "AMICI" ("ID_Utente", "ID_Amico") o "APPARTIENE_A" ("ID_Attivita", "ID_Categoria"), non sono presenti attributi descrittivi che dipendono solo da uno dei due ID, garantendo la piena dipendenza funzionale
- **Terza forma normale** (**3NF**): un database è in 3NF se è in 2NF e se non esistono dipendenze transitive, ovvero attributi che dipendono dalla chiave primaria attraverso altri attributi non-chiave:
    - *Decomposizione delle informazioni*: i dati sono stati segmentati in modo che ogni attributo descriva solo l'entità identificata

dalla chiave; gli attributi opzionali (come "Wi-Fi" o "Parcheggio") e le categorie sono gestiti in tabelle separate ("ATTRIBUTO", "CATEGORIA"), impedendo che il nome di un attributo dipenda direttamente dall'ID dell'attività commerciale.

La progettazione logica è stata inoltre corredata da una serie di vincoli accessori allo scopo di garantire l'integrità dei dati:

- Controlli su *Stato_apertura*, in modo che contenga solo i valori 0 e 1;
- Controlli su *Latitudine* e *Longitudine*, in modo che i valori siano compresi rispettivamente tra -90 e 90 e -180 e 180;
- Controlli sulle *Stelle* nelle recensioni e nelle attività, in modo che sia un valore compreso tra 1 e 5;

2.4 Progettazione fisica

La progettazione fisica è l'ultimo stadio di progettazione. Se la progettazione logica ha definito "quali tabelle creare", la progettazione fisica decide "come scriverle sul disco fisso" per ottenere le massime prestazioni; in questa fase, il modello diventa codice reale e si adatta alle caratteristiche specifiche del software scelto

Vediamo quindi di seguito i comandi in DDL (Data Definition Language), ovvero il sottocomponente del linguaggio SQL utilizzata per definire, modificare o eliminare la struttura degli oggetti all'interno del database, come tabelle e vincoli:

Tabella Utente

```
-- CREAZIONE TABELLE
CREATE TABLE  "UTENTE" (
    "ID_Utente" VARCHAR2(22) DEFAULT SUBSTR(RAWTOHEX(SYS_GUID()), 1, 22)
NOT NULL ENABLE,
    "Nome" VARCHAR2(50) NOT NULL ENABLE,
    "Data_iscrizione" DATE NOT NULL ENABLE,
    "Num_recensioni" NUMBER(*,0) NOT NULL ENABLE,
    "Num_cool" NUMBER(*,0) NOT NULL ENABLE,
    "Num_funny" NUMBER(*,0) NOT NULL ENABLE,
    "Num_useful" NUMBER(*,0) NOT NULL ENABLE,
    "Media_recensioni" NUMBER NOT NULL ENABLE,
     CONSTRAINT "PK_UTENTE" PRIMARY KEY ("ID_Utente")
)
```

Tabella Attività

```
CREATE TABLE  "ATTIVITA" (
    "ID_Attivita" VARCHAR2(22) DEFAULT SUBSTR(RAWTOHEX(SYS_GUID()), 1,
22) NOT NULL ENABLE,
    "Nome" VARCHAR2(50) NOT NULL ENABLE,
    "Indirizzo" VARCHAR2(50) NOT NULL ENABLE,
    "Citta" VARCHAR2(50) NOT NULL ENABLE,
    "Stato" VARCHAR2(50) NOT NULL ENABLE,
    "CAP" VARCHAR2(10) NOT NULL ENABLE,
    "Latitudine" FLOAT(126) NOT NULL ENABLE,
    "Longitudine" FLOAT(126) NOT NULL ENABLE,
    "Stelle" NUMBER(3,2) NOT NULL ENABLE,
    "Num_recensioni" NUMBER(*,0) DEFAULT 0 NOT NULL ENABLE,
    "Stato_apertura" NUMBER(1,0) DEFAULT 0 NOT NULL ENABLE,
        CONSTRAINT "BOOL_STATO_APERTURA" CHECK ("Stato_apertura" IN
(0,1)) ENABLE,
        CONSTRAINT "PK_ATTIVITA" PRIMARY KEY ("ID_Attivita"),
        CONSTRAINT "CHECK_LIMITE_STELLE_ATTIVITA" CHECK ("Stelle" >= 1
AND "Stelle" <= 5) ENABLE,
        CONSTRAINT "CHECK_LAT" CHECK ("Latitudine" BETWEEN -90 AND 90)
ENABLE,
        CONSTRAINT "CHECK_LON" CHECK ("Longitudine" BETWEEN -180 AND 180)
ENABLE
)
```

Tabella Recensione

```
CREATE TABLE  "RECENSIONE" (
    "ID_Recensione" VARCHAR2(22) DEFAULT SUBSTR(RAWTOHEX(SYS_GUID()), 1,
22) NOT NULL ENABLE,
    "Stelle" NUMBER(*,0) NOT NULL ENABLE,
    "Data" DATE NOT NULL ENABLE,
    "Testo" VARCHAR2(3000),
    "Num_cool" NUMBER(*,0) DEFAULT 0 NOT NULL ENABLE,
    "Num_funny" NUMBER(*,0) DEFAULT 0 NOT NULL ENABLE,
    "Num_useful" NUMBER(*,0) DEFAULT 0 NOT NULL ENABLE,
    "ID_Utente" VARCHAR2(22) NOT NULL ENABLE,
    "ID_Attivita" VARCHAR2(22) NOT NULL ENABLE,
        CONSTRAINT "PK_RECENSIONE" PRIMARY KEY ("ID_Recensione"),
        CONSTRAINT "CHECK_LIMITE_STELLE_RECENSIONE" CHECK ("Stelle" >= 1
AND "Stelle" <= 5) ENABLE
)
```

Tabella Amici

```
CREATE TABLE  "AMICI" (
    "ID_Utente" VARCHAR2(22),
    "ID_Amico" VARCHAR2(22),
        CONSTRAINT "PK_AMICI" PRIMARY KEY ("ID_Utente", "ID_Amico")
)
```

Tabella Check-in

```
CREATE TABLE  "CHECKIN" (
    "ID_Attivita" VARCHAR2(22),
    "ID_Utente" VARCHAR2(22),
    "Timestamp" DATE,
        CONSTRAINT "PK_CHECKIN" PRIMARY KEY ("ID_Attivita", "ID_Utente",
"Timestamp")
)
```

Tabella Foto

```
CREATE TABLE  "FOTO" (
    "ID_Foto" VARCHAR2(22) DEFAULT SUBSTR(RAWTOHEX(SYS_GUID()), 1, 22)
NOT NULL ENABLE,
    "Didascialia" VARCHAR2(3000),
    "Etichetta" VARCHAR2(200),
    "ID_Utente" VARCHAR2(22) NOT NULL ENABLE,
    "ID_Attivita" VARCHAR2(22) NOT NULL ENABLE,
        CONSTRAINT "PK_FOTO" PRIMARY KEY ("ID_Foto")
)
```

Tabella Suggerimento

```
CREATE TABLE  "SUGGERIMENTO" (
    "Testo" VARCHAR2(3000),
    "Data_caricamento" DATE,
    "Num_complimenti" NUMBER(*,0) DEFAULT 0,
    "ID_Utente" VARCHAR2(22) NOT NULL ENABLE,
    "ID_Attivita" VARCHAR2(22) NOT NULL ENABLE,
        CONSTRAINT "PK_SUGGERIMENTO" PRIMARY KEY ("ID_Utente",
"ID_Attivita", "Data_caricamento")
)
```

Tabella Categoria

```
CREATE TABLE  "CATEGORIA" (
    "ID_Categoria" VARCHAR2(22) DEFAULT SUBSTR(RAWTOHEX(SYS_GUID()), 1,
22) NOT NULL ENABLE,
    "Nome" VARCHAR2(50) NOT NULL ENABLE,
        CONSTRAINT "PK_CATEGORIA" PRIMARY KEY ("ID_Categoria")
)
```

Tabella Appartiene_A

```
CREATE TABLE  "APPARTIENE_A" (
    "ID_Attivita" VARCHAR2(22),
    "ID_Categoria" VARCHAR2(22),
        CONSTRAINT "PK_APPARTIENE" PRIMARY KEY ("ID_Attivita",
"ID_Categoria")
)
```

Tabella Orario

```
CREATE TABLE  "ORARIO" (
    "ID_Orario" VARCHAR2(22) DEFAULT SUBSTR(RAWTOHEX(SYS_GUID()), 1, 22)
NOT NULL ENABLE,
    "Giorno" VARCHAR2(10) NOT NULL ENABLE,
    "Ora_apertura" TIMESTAMP NOT NULL ENABLE,
    "Ora_chiusura" TIMESTAMP NOT NULL ENABLE,
    "ID_Attivita" VARCHAR2(22),
        CONSTRAINT "PK_ORARIO" PRIMARY KEY ("ID_Orario")
)
```

Tabella Attributo

```
CREATE TABLE  "ATTRIBUTO" (
    "ID_Attributo" VARCHAR2(22) DEFAULT SUBSTR(RAWTOHEX(SYS_GUID()), 1,
22) NOT NULL ENABLE,
    "Nome" VARCHAR2(50) NOT NULL ENABLE,
        CONSTRAINT "PK_ATTRIBUTO" PRIMARY KEY ("ID_Attributo")
)
```

Tabella Possiede

```
CREATE TABLE  "POSSIEDE" (
    "ID_Attivita" VARCHAR2(22),
    "ID_Attributo" VARCHAR2(22),
        CONSTRAINT "PK_OFFRE" PRIMARY KEY ("ID_Attivita", "ID_Attributo")
)
```

Alter table per la definizione delle chiavi esterne

```
-- Collegamenti Tabella RECENSIONE
ALTER TABLE "RECENSIONE" ADD CONSTRAINT "FK_RECENSIONE_UTENTE" FOREIGN
KEY ("ID_Utente")
      REFERENCES "UTENTE" ("ID_Utente") ENABLE;

ALTER TABLE "RECENSIONE" ADD CONSTRAINT "FK_RECENSIONE_ATTIVITA" FOREIGN
KEY ("ID_Attivita")
      REFERENCES "ATTIVITA" ("ID_Attivita") ENABLE;

-- Collegamenti Tabella AMICI
```

```
ALTER TABLE "AMICI" ADD CONSTRAINT "FK_AMICI_UTENTE" FOREIGN KEY
("ID_Utente")
      REFERENCES "UTENTE" ("ID_Utente") ENABLE;

ALTER TABLE "AMICI" ADD CONSTRAINT "FK_AMICI_AMICO" FOREIGN KEY
("ID_Amico")
      REFERENCES "UTENTE" ("ID_Utente") ENABLE;

-- Collegamenti Tabella CHECKIN
ALTER TABLE "CHECKIN" ADD CONSTRAINT "FK_CHECKIN_ATTIVITA" FOREIGN KEY
("ID_Attivita")
      REFERENCES "ATTIVITA" ("ID_Attivita") ENABLE;

ALTER TABLE "CHECKIN" ADD CONSTRAINT "FK_CHECKIN_UTENTE" FOREIGN KEY
("ID_Utente")
      REFERENCES "UTENTE" ("ID_Utente") ENABLE;

-- Collegamenti Tabella FOTO
ALTER TABLE "FOTO" ADD CONSTRAINT "FK_FOTO_UTENTE" FOREIGN KEY
("ID_Utente")
      REFERENCES "UTENTE" ("ID_Utente") ENABLE;

ALTER TABLE "FOTO" ADD CONSTRAINT "FK_FOTO_ATTIVITA" FOREIGN KEY
("ID_Attivita")
      REFERENCES "ATTIVITA" ("ID_Attivita") ENABLE;

-- Collegamenti Tabella SUGGERIMENTO
ALTER TABLE "SUGGERIMENTO" ADD CONSTRAINT "FK_SUGGERIMENTO_UTENTE"
FOREIGN KEY ("ID_Utente")
      REFERENCES "UTENTE" ("ID_Utente") ENABLE;

ALTER TABLE "SUGGERIMENTO" ADD CONSTRAINT "FK_SUGGERIMENTO_ATTIVITA"
FOREIGN KEY ("ID_Attivita")
      REFERENCES "ATTIVITA" ("ID_Attivita") ENABLE;

-- Collegamenti Tabella APPARTIENE_A
ALTER TABLE "APPARTIENE_A" ADD CONSTRAINT "FK_APPARTIENE_ATTIVITA"
FOREIGN KEY ("ID_Attivita")
      REFERENCES "ATTIVITA" ("ID_Attivita") ENABLE;

ALTER TABLE "APPARTIENE_A" ADD CONSTRAINT "FK_APPARTIENE_CATEGORIA"
FOREIGN KEY ("ID_Categoria")
      REFERENCES "CATEGORIA" ("ID_Categoria") ENABLE;

-- Collegamenti Tabella ORARIO
ALTER TABLE "ORARIO" ADD CONSTRAINT "FK_ORARIO_ATTIVITA" FOREIGN KEY
("ID_Attivita")
```

```
      REFERENCES "ATTIVITA" ("ID_Attivita") ENABLE;

-- Collegamenti Tabella POSSIEDE
ALTER TABLE "POSSIEDE" ADD CONSTRAINT "FK_POSSIEDE_ATTIVITA" FOREIGN KEY
("ID_Attivita")
      REFERENCES "ATTIVITA" ("ID_Attivita") ENABLE;

ALTER TABLE "POSSIEDE" ADD CONSTRAINT "FK_POSSIEDE_ATTRIBUTO" FOREIGN KEY
("ID_Attributo")
      REFERENCES "ATTRIBUTO" ("ID_Attributo") ENABLE;
```

Dal punto di vista del **dimensionamento fisico** del database, di seguito sono presentate le dimensioni delle tabelle create:

Tabella Utente

Attributo	**Data type**	**Dimensione (byte)**
ID_Utente	VARCHAR2(22)	22
Nome	VARCHAR2(50)	50
Data_iscrizione	DATE	7
Num_recensioni	NUMBER(*,0)	22
Num_cool	NUMBER(*,0)	22
Num_funny	NUMBER(*,0)	22
Num_useful	NUMBER(*,0)	22
Media_recensioni	NUMBER	22
Totale:		**200**

Tabella Attività

Attributo	**Data type**	**Dimensione (byte)**
ID_Attivita	VARCHAR2(22)	22
Nome	VARCHAR2(50)	50
Indirizzo	VARCHAR2(50)	50
Citta	VARCHAR2(50)	50
Stato	VARCHAR2(50)	50
CAP	VARCHAR2(10)	10
Latitudine	FLOAT(126)	22
Longitudine	FLOAT(126)	22
Stelle	NUMBER(3,2)	3
Num_recensioni	NUMBER(*,0)	22
Stato_apertura	NUMBER(1,0)	2
Totale:		**317**

Tabella Recensione

Attributo	**Data type**	**Dimensione (byte)**
ID_Recensione	VARCHAR2(22)	22
Stelle	NUMBER(3,2)	3
Data	DATE	7
Testo	VARCHAR2(3000)	3000
Num_cool	NUMBER(*,0)	22
Num_funny	NUMBER(*,0)	22
Num_useful	NUMBER(*,0)	22
ID_Utente	VARCHAR2(22)	22
ID_Attivita	VARCHAR2(22)	22
Totale:		**3156**

Tabella Amici

Attributo	**Data type**	**Dimensione (byte)**
ID_Utente	VARCHAR2(22)	22
ID_Amico	VARCHAR2(22)	22
Totale:		**49**

Tabella Check-In

Attributo	**Data type**	**Dimensione (byte)**
ID_Attivita	VARCHAR2(22)	22
ID_Utente	VARCHAR2(22)	22
Timestamp	DATE	7
Totale:		**57**

Tabella Foto

Attributo	**Data type**	**Dimensione (byte)**
ID_Foto	VARCHAR2(22)	22
Didascalia	VARCHAR2(3000)	3000
Erichetta	VARCHAR2(200)	200
ID_Utente	VARCHAR2(22)	22
ID_Attivita	VARCHAR2(22)	22
Totale:		**3276**

Tabella Suggerimento

Attributo	**Data type**	**Dimensione (byte)**
Testo	VARCHAR2(3000)	3000
Data_caricamento	DATE	7
Num_complimenti	NUMBER(*,0)	22
ID_Utente	VARCHAR2(22)	22
ID_Attivita	VARCHAR2(22)	22
Totale:		**3084**

Tabella Categoria

Attributo	**Data type**	**Dimensione (byte)**
ID_Categoria	VARCHAR2(22)	22
Nome	VARCHAR2(22)	22
Totale:		**49**

Tabella Appartiene_A

Attributo	**Data type**	**Dimensione (byte)**
ID_Attivita	VARCHAR2(22)	22
ID_Categoria	VARCHAR2(22)	22
Totale:		**49**

Tabella Orario

Attributo	**Data type**	**Dimensione (byte)**
ID_Orario	VARCHAR2(22)	22
Giorno	VARCHAR2(10)	10
Ora_aperuta	TIMESTAMP	7
Ora_chiusura	TIMESTAMP	7
ID_Attivita	VARCHAR2(22)	22
Totale:		**76**

Tabella Attributo

Attributo	**Data type**	**Dimensione (byte)**
ID_Attributo	VARCHAR2(22)	22
Nome	VARCHAR2(50)	50
Totale:		**82**

Tabella Possiede

Attributo	**Data type**	**Dimensione (byte)**
ID_Attivita	VARCHAR2(22)	22
ID_Attributo	VARCHAR2(22)	22
Totale:		**49**

2.5 Pulizia e caricamento del datased

Una nota sul nostro caso operativo è che il datased di riferimento per il progetto presenta diverse lacune che spaziano da campi vuoti a lista non formattate correttamente; pertanto, prima di procedere al caricamento sul DB è stato necessario effettuare alcune operazioni pulizia attraverso la libreria di Python “Pandas”.

Di seguito un esempio di codice di pulizia, nel caso della tabella review:

```
def convert_yelp_json_to_csv(input_json, output_csv):
    print(f"Caricamento e pulizia di {input_json}...")

    # Caricamento a blocchi per gestire file di grandi dimensioni
    chunks = pd.read_json(input_json, lines=True, chunksize=10000)

    first_chunk = True
    for df in chunks:
        # 1. Rimozione record con review_id o stars vuoti
        df = df.dropna(subset=['review_id', 'stars'])
        df = df[df['review_id'].astype(str).str.strip() != '']

        # 2. Pulizia e Troncamento del testo (max 3000 caratteri)
        if 'text' in df.columns:
            # Rimuoviamo i ritorni a capo righe nel CSV/SQL
            df['text'] = df['text'].str.replace(r'[\n\r]+', ' ',
regex=True)
            # Tronca a 3000 caratteri
            df['text'] = df['text'].str.slice(0, 3000)

        # 3. Validazione specifica degli ID (deve essere di 22 caratteri)
        df = df[df['review_id'].str.len() == 22]

        # 4. Conversione tipi per coerenza con lo schema SQL
        df['stars'] = df['stars'].astype(int)
        for col in ['useful', 'funny', 'cool']:
            if col in df.columns:
                df[col] = df[col].fillna(0).astype(int)

        # Scrittura su CSV con quoting protettivo
        df.to_csv(output_csv,
                  mode='a',
                  index=False,
                  header=first_chunk,
                  sep=',',
                  encoding='utf-8',
                  quoting=1) # 1 = csv.QUOTE_ALL

        first_chunk = False
        print(f"Processate {len(df)} righe valide...")

    print(f"Conversione completata! File salvato come: {output_csv}")

# Esecuzione
convert_yelp_json_to_csv('review.json', 'recensioni_finali.csv')
```

3. Ottimizzazione DB

Una volta che un database è stato progettato e popolato, la sfida si sposta dalla struttura alle prestazioni e alla sicurezza. Un database "perfetto" sulla carta può rivelarsi inutilizzabile se non è in grado di rispondere rapidamente alle interrogazioni degli utenti o se non è protetto da guasti improvvisi.

Vediamo quindi una serie di tecnologie che un buon database implementa per garantire un servizio efficiente ed affidabile.

3.1 Indici

Un indice è una struttura dati accessoria che permette al motore di ricerca (Query Optimizer) di trovare le righe di una tabella senza doverle scansionare una per una (Full Table Scan). Questo permette di ridurre drasticamente il Query Response Time, fattore essenziali per sistemi real time con cui gli utenti interagiscono giornalmente. In particolare, abbiamo:

- indici su chiavi esterne: vitali per evitare scansioni complete delle tabelle durante le join e per prevenire problemi di lock sulla tabella padre quando si eliminano record dalla tabella figlia;
- indici per ricerche testuali (Lookups): fondamentali per l'interazione utente per rendere veloce le ricerche tramite testo;
- indici per ordinamento temporale: fondamentali per ordinare temporalmente i dati, nel campo di interesse, infatti, le informazioni più recenti sono considerati più attendibili;

Vediamo quindi come sono state implementate in DDL:

```sql
--INDICI PER MIGLIORARE LA PERFORMANCE
-- Indici per la tabella RECENSIONE (Join frequenti tra Utenti e
Attivita')
CREATE INDEX IDX_FK_REC_UTENTE ON RECENSIONE ("ID_Utente");
CREATE INDEX IDX_FK_REC_ATTIVITA ON RECENSIONE ("ID_Attivita");

-- Indici per la tabella AMICI (Velocizza la ricerca di chi e' amico di
un dato utente)
CREATE INDEX IDX_FK_AMICI_AMICO ON AMICI ("ID_Amico");

-- Indici per i contenuti multimediali (FOTO)
CREATE INDEX IDX_FK_FOTO_UTENTE ON FOTO ("ID_Utente");
CREATE INDEX IDX_FK_FOTO_ATTIVITA ON FOTO ("ID_Attivita");

-- Indice per gli orari delle attivita'
CREATE INDEX IDX_FK_ORARIO_ATTIVITA ON ORARIO ("ID_Attivita");

-- Indici per le tabelle di associazione (N:M)
CREATE INDEX IDX_FK_APPARTIENE_CAT ON APPARTIENE_A ("ID_Categoria");
CREATE INDEX IDX_FK_POSSIEDE_ATTR ON POSSIEDE ("ID_Attributo");

-- Ricerca rapida di un'attivita' per nome
CREATE INDEX IDX_ATTIVITA_NOME ON ATTIVITA ("Nome");

-- Ricerca rapida di un utente per nome
CREATE INDEX IDX_UTENTE_NOME ON UTENTE ("Nome");

-- Per visualizzare le ultime recensioni pubblicate
CREATE INDEX IDX_RECENSIONE_DATA_DESC ON RECENSIONE ("Data" DESC);

-- Per visualizzare gli ultimi suggerimenti caricati
CREATE INDEX IDX_SUGGERIMENTO_DATA_DESC ON SUGGERIMENTO
("Data_caricamento" DESC);
```

3.2 Gestione della concorrenza

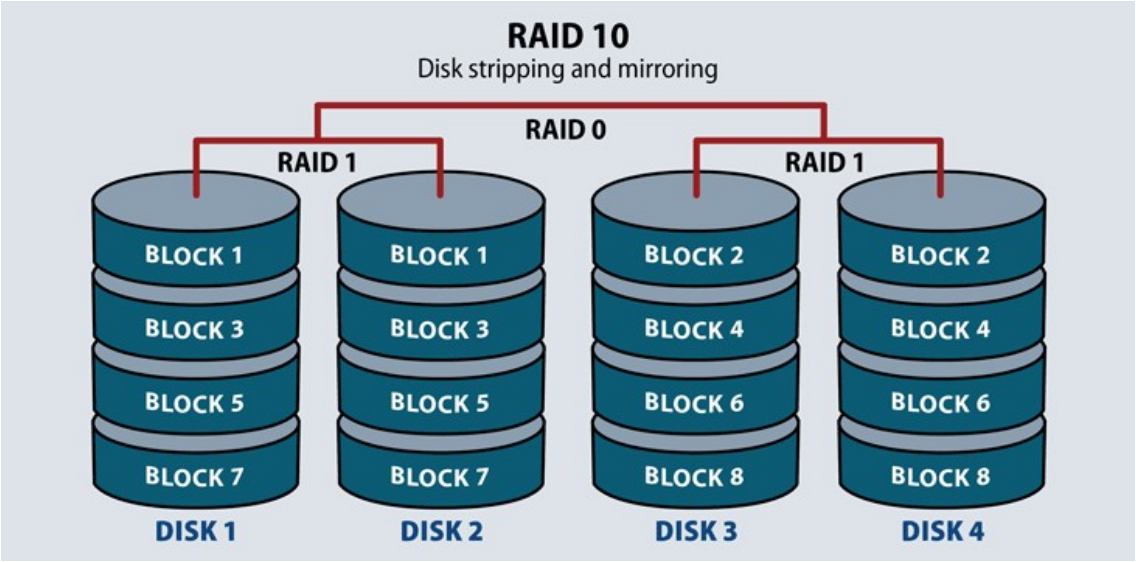
La nostra realtà di interesse prevende milioni di utenti che interagiscono contemporaneamente su diverse tabelle per pubblicare recensioni, foto, check-in ed altro ancora. Per evitare possibilità di perdite di aggiornamento, letture sporche o inconsistenti e aggiornamenti fantasma è necessario adottare le opportune politiche di gestione della concorrenza.

Uno dei metodi di gestione delle occorrenze più utilizzato sui sistemi real time è il **Locking a due fasi stretto** (o Strict two phase locking o 2PL); questa tecnica richiede che ogni transazione sul database acquisisca mediante richieste di *read_lock* e *write_lock*, in una prima fase detta crescente, tutti i lock sugli oggetti su cui dovrà fare operazioni lettura o scrittura. I lock, infatti, sono meccanismi di sincronizzazione che limitano l'accesso ad una risorsa condivisa ad una sola o più transazioni alla volta.

Una volta terminate tutte le operazioni, la transazione rilascerà, in una seconda fase detta decrescente, tutti i lock precedentemente acquisiti mediante primitive di *unlock*. Nella variante stretta, in particolare, i lock possono essere rilasciati solo al termine dei commit o abort.

3.3 Gestione dell'affidabilità

Il sistema oltre ad essere pronto a gestire l'accesso multiplo al database deve essere anche in grado di ridurre al minimo i rischi di perdita dei dati; per rendere ciò possibile esistono diverse strategie di conservazione dei dati, nel nostro caso, vista l'enorme mole di accessi contemporanei e di dati da conservare si è scelto il **RAID 10**.



Il RAID 10 è una combinazione dei livelli RAID 1 e 0 dove diversi sistemi RAID 1 formano un sistema RAID 0.

Il principio alla base del RAID 1 serve a rendere possibile la replica (**mirroring**) dei file all'interno della concatenazione RAID 10. Tutti i file sono archiviati in duplice versione così da garantire piena ridondanza.

A questo si aggiunge anche lo **striping** dei dati: i blocchi di dati vengono scomposti in singole strisce (stripe) e archiviati sui supporti concatenati nel RAID 10. Gli hard disk che formano uno dei RAID 1 che compongono il sistema dispongono perciò sempre dello stesso set di dati, diverso però da quello degli altri cluster RAID 1.

La combinazione tra mirroring e striping ci permette quindi di avere un sistema sicuro dai guasti ma al contempo veloce nell'accesso ai dati.

3.4 Gestione della recovery

Nonostante il sistema sia progettato per resistere ai guasti e continuare ad operare ininterrottamente, è necessario comunque implementare delle tecnologie di recovery per far tornare il sistema ad operare a seguito di crash.

Il cardine fondamentale della recovery è il **file di log**: un log è un file presente sulla memoria stabile che registra tutte le operazioni svolte dalle transazioni nel loro ordine di esecuzione. Spesso viene affiancato da *checkpoint* e *dump* che ci consentono di ricostruire gli istanti prima del crash e attuare le giuste politiche ti ripristino.

In particolare, distinguiamo:

- Ripresa a caldo: attuato in caso di guasti soft in cui si perde solo il contenuto della memoria centrale; si individua dal checkpoint l'ultima transazione effettuata correttamente, dopodiché si ripercorre il log all'indietro tramite operazioni di undo e poi in avanti con operazioni di redo;

- Ripresa a freddo: attuato in caso di guasti hard in cui si perde anche il contenuto della memoria stabile; si individua dal dumo il più recente record di log e si eseguono tutte le operazioni registrate relative alla parte di memoria degradata operando sui dati provenienti dai dischi di backup.

3.5 Gestione della sicurezza

Nelle specifiche di progetto ci viene esplicitamente richiesto di attuare misure di sicurezza per gestire diversi tipi di utenti e i relativi permessi.

La creazione di utenti diversificati permette di tutelare l'integrità del database e rispettare i principi di responsabilità e minimizzazione degli accessi ai dati sensibili.

Di seguito le operazioni in DDL per creare le classi di utenti e i loro grant.

Utente amministratore

```
-- Creazione Amministratore
CREATE USER "AMMINISTRATORE_DB" IDENTIFIED BY "passamministratore";
-- Assegnazione Privilegi Completi
GRANT ALL PRIVILEGES TO "AMMINISTRATORE_DB";
```

Utente moderatore

```
-- Creazione Moderatore
CREATE USER "MODERATORE_DB" IDENTIFIED BY "passmoderatore";
-- Privilegi di gestione sui contenuti generati dagli utenti
GRANT SELECT, UPDATE, DELETE ON "RECENSIONE" TO "MODERATORE_DB";
GRANT SELECT, UPDATE, DELETE ON "SUGGERIMENTO" TO "MODERATORE_DB";
GRANT SELECT, UPDATE, DELETE ON "FOTO" TO "MODERATORE_DB";
GRANT SELECT, DELETE ON "CHECKIN" TO "MODERATORE_DB";
-- Privilegi di sola lettura per consultazione profili e attivita'
GRANT SELECT ON "UTENTE" TO "MODERATORE_DB";
GRANT SELECT ON "ATTIVITA" TO "MODERATORE_DB";
```

Utente base

```
-- Creazione Utente
CREATE USER "UTENTE_DB" IDENTIFIED BY "passutente";
-- Privilegi di consultazione generale
GRANT SELECT ON "ATTIVITA" TO "UTENTE_DB";
GRANT SELECT ON "CATEGORIA" TO "UTENTE_DB";
GRANT SELECT ON "ORARIO" TO "UTENTE_DB";
GRANT SELECT ON "ATTRIBUTO" TO "UTENTE_DB";
-- Privilegi di inserimento e modifica (solo sui contenuti con il loro
ID)
GRANT INSERT, UPDATE, DELETE ON "RECENSIONE" TO "UTENTE_DB";
GRANT INSERT, UPDATE, DELETE ON "SUGGERIMENTO" TO "UTENTE_DB";
GRANT INSERT, UPDATE, DELETE ON "FOTO" TO "UTENTE_DB";
GRANT INSERT ON "CHECKIN" TO "UTENTE_DB";
-- Privilegi per interazioni sociali
GRANT INSERT, DELETE ON "AMICI" TO "UTENTE_DB";
```

4. Funzionalità DB

Una volta stabilite tutte le politiche di ottimizzazione del database, andiamo ora a definire tutti quei meccanismi che rendono il nostro database veramente dinamico e facilmente utilizzabile sia da amministratori che da utenti abituali.

4.1 Trigger

Un trigger è una specifica per la quale una data azione o funzione deve attivarsi in maniera automatica, ogni qual volta un'operazione stabilita viene eseguita su un oggetto specificato della base di dati. Si fonda sul paradigma E-C-A. Ovvero:

- *Evento*: una primitiva per la manipolazione dei dati
- *Condizione*: predicato booleano atto a valutare il verificarsi di un evento
- *Azione*: sequenza di operazioni da eseguire quando i due elementi precedenti hanno esito positivo.

Sono stati realizzati i seguenti trigger:

Trigger 1: modifica di Num_recensioni e Media_recensioni nella tabella Utente ad ogni aggiunta/modifica/eliminazione di una recensione

```
--Modifica Num_recensioni e Media_recensioni in Utente ad ogni
aggiunta/modifica/eliminazione di una recensione
CREATE OR REPLACE TRIGGER "TRG_AGGIORNA_STATISTICHE_UTENTE"
AFTER INSERT OR UPDATE OR DELETE ON "RECENSIONE"
FOR EACH ROW
DECLARE
    v_id_utente VARCHAR2(22);
BEGIN
    -- Identifica l'ID utente corretto (OLD per cancellazioni, NEW per
inserimenti/modifiche)
    IF DELETING THEN
        v_id_utente := :OLD."ID_Utente";
    ELSE
        v_id_utente := :NEW."ID_Utente";
    END IF;

    -- Aggiornamento della valutazione media e del numero totale di
recensioni dell'utente
    UPDATE "UTENTE"
    SET "Media_recensioni" = (
        SELECT NVL(AVG("Stelle"), 0)
        FROM "RECENSIONE"
        WHERE "ID_Utente" = v_id_utente
    ),
    "Num_recensioni" = (
        SELECT COUNT(*)
        FROM "RECENSIONE"
        WHERE "ID_Utente" = v_id_utente
    )
    WHERE "ID_Utente" = v_id_utente;
END;
```

Trigger 2: modifica di Num_recensioni e Stelle nella tabella Attività ad ogni aggiunta/modifica/eliminazione di una recensione

```
--Modifica Num_recensioni e Stelle in Attivita' ad ogni
aggiunta/modifica/eliminazione di una recensione
CREATE OR REPLACE TRIGGER "TRG_AGGIORNA_STATISTICHE_ATTIVITA"
AFTER INSERT OR UPDATE OR DELETE ON "RECENSIONE"
FOR EACH ROW
DECLARE
    v_id_attivita VARCHAR2(22);
BEGIN
    -- Identifichiamo l'ID dell'attività corretto (OLD per DELETE, NEW
per INSERT/UPDATE)
    IF DELETING THEN
        v_id_attivita := :OLD."ID_Attivita";
    ELSE
        v_id_attivita := :NEW."ID_Attivita";
    END IF;

    -- Aggiornamento automatico della valutazione media e del numero
totale di recensioni
    UPDATE "ATTIVITA"
    SET "Stelle" = (
        SELECT NVL(AVG("Stelle"), 0)
        FROM "RECENSIONE"
        WHERE "ID_Attivita" = v_id_attivita
    ),
    "Num_recensioni" = (
        SELECT COUNT(*)
        FROM "RECENSIONE"
        WHERE "ID_Attivita" = v_id_attivita
    )
    WHERE "ID_Attivita" = v_id_attivita;
END;
```

Trigger 3: rimozione relazione speculare alla rimozione di una amicizia

```
CREATE OR REPLACE TRIGGER "TRG_RIMUOVI_AMICIZIA_SIMMETRICA"
AFTER DELETE ON "AMICI"
FOR EACH ROW
BEGIN
    -- Elimina la relazione speculare (Amico -> Utente)
    DELETE FROM "AMICI"
    WHERE "ID_Utente" = :OLD."ID_Amico"
      AND "ID_Amico" = :OLD."ID_Utente";
END;
```

4.2 Stored procedure

Una stored procedure è un blocco di codice PL/SQL (Procedural Language/SQL) che viene nominato, compilato e memorizzato direttamente all'interno del database. A differenza dei trigger, devono essere richiamati esplicitamente.

Sono state realizzate le seguenti stored procedure.

SP 1: eliminare ogni traccia di un utente dal DB

```
--SP per eliminare ogni traccia di un utente dal DB
CREATE OR REPLACE PROCEDURE "Elimina_Utente_Completo" (
    p_ID_Utente IN VARCHAR2
) AS
BEGIN
    -- 1. Eliminazione dalle relazioni di amicizia (sia come utente che
come amico)
    DELETE FROM "AMICI"
    WHERE "ID_Utente" = p_ID_Utente OR "ID_Amico" = p_ID_Utente;

    -- 2. Eliminazione dei check-in registrati dall'utente
    DELETE FROM "CHECKIN"
    WHERE "ID_Utente" = p_ID_Utente;

    -- 3. Eliminazione delle fotografie caricate dall'utente
    DELETE FROM "FOTO"
    WHERE "ID_Utente" = p_ID_Utente;

    -- 4. Eliminazione dei suggerimenti rapidi pubblicati dall'utente
    DELETE FROM "SUGGERIMENTO"
    WHERE "ID_Utente" = p_ID_Utente;

    -- 5. Eliminazione delle recensioni scritte dall'utente
    DELETE FROM "RECENSIONE"
    WHERE "ID_Utente" = p_ID_Utente;

    -- 6. Infine, eliminazione del profilo utente principale
    DELETE FROM "UTENTE"
    WHERE "ID_Utente" = p_ID_Utente;

    COMMIT;
END;
```

SP 2: correggere stelle e testo di una recensione

```
--SP per correggere stelle e testo di una recensione
CREATE OR REPLACE PROCEDURE "Correggi_Recensione" (
    p_ID_Recensione IN VARCHAR2,
    p_Nuove_Stelle IN NUMBER,
    p_Nuovo_Testo IN VARCHAR2
) AS
    v_ID_Attivita VARCHAR2(22);
BEGIN
    -- 1. Recupero l'ID dell'attivita' associata alla recensione per
aggiornare la media in seguito
    SELECT "ID_Attivita" INTO v_ID_Attivita
    FROM "RECENSIONE"
    WHERE "ID_Recensione" = p_ID_Recensione;

    -- 2. Aggiornamento del testo e delle stelle nella tabella RECENSIONE
    -- Il database verifichera' automaticamente il vincolo
CHECK_LIMITE_STELLE_RECENSIONE
    UPDATE "RECENSIONE"
    SET "Stelle" = p_Nuove_Stelle,
        "Testo" = p_Nuovo_Testo,
        "Data" = SYSDATE -- Opzionale: aggiorna la data all'ultimo
intervento
    WHERE "ID_Recensione" = p_ID_Recensione;

    COMMIT;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Errore: ID Recensione non trovato.');
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END;
```

SP 3: modificare didascalia di una foto

```
--SP per modificare la didascalia di una foto
CREATE OR REPLACE PROCEDURE "Cambia_Didascalia_Foto" (
    p_ID_Foto IN VARCHAR2,
    p_Nuova_Didascalia IN VARCHAR2
) AS
BEGIN
    -- Aggiornamento della didascalia per la foto identificata dall'ID
univoco
    UPDATE "FOTO"
    SET "Didascialia" = p_Nuova_Didascalia
    WHERE "ID_Foto" = p_ID_Foto;

    COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        -- In caso di errore imprevisto, annulla l'operazione
        ROLLBACK;
        RAISE;
END;
```

4.3 Viste

Una Vista è come una "tabella virtuale". A differenza di una tabella reale, una vista non contiene dati propri e non occupa spazio fisico su disco (se non per la memorizzazione della sua definizione nel dizionario dati).

Sono state realizzate le seguenti viste:

Vista 1: recensioni senza didascalia

```
--Vista delle recensioni senza didascalia
CREATE VIEW "V_FOTO_SENZA_ETICHETTA" AS
SELECT "ID_Foto", "Etichetta", "ID_Utente", "ID_Attivita"
FROM "FOTO"
WHERE "Didascialia" IS NULL;
```

Vista 2: recensioni caricate nell'ultimo mese

```
--Vista recensioni caricate nell'ultimo mese
CREATE VIEW "V_RECENSIONI_ULTIMO_MESE" AS
SELECT *
FROM "RECENSIONE"
WHERE "Data" >= SYSDATE - 30;
```

Vista 3: orari delle attività aperte il lunedì

```
--Orari delle attivita' aperte il lunedi'
CREATE VIEW "V_ORARI_LUNEDI" AS
SELECT a."Nome", o."Ora_apertura", o."Ora_chiusura", a."Citta"
FROM "ORARIO" o
JOIN "ATTIVITA" a ON o."ID_Attivita" = a."ID_Attivita"
WHERE o."Giorno" = 'Monday';
```

4.4 Query

Una query è un'istruzione inviata al database per interagire con i dati. Le query sono scritte in DQL (Data Query Language)

Sono state realizzate le seguenti query:

Query 1: migliori 10 attività per valutazione media più alta in una città

```
--Attivita' con la valutazione media piu' alta in una citta'
SELECT "Nome", "Indirizzo", "Stelle", "Num_recensioni"
FROM ATTIVITA
WHERE "Citta" = 'Tampa'
ORDER BY "Stelle" DESC, "Num_recensioni" DESC
FETCH FIRST 10 ROWS ONLY;
```

Query 2: numero medio di recensioni per categoria

```
--Numero medio di recensioni per categoria
SELECT c."Nome" AS "Categoria", ROUND(AVG(a."Num_recensioni"), 2) AS
"Media_Recensioni"
FROM "CATEGORIA" c
JOIN "APPARTIENE_A" aa ON c."ID_Categoria" = aa."ID_Categoria"
JOIN "ATTIVITA" a ON aa."ID_Attivita" = a."ID_Attivita"
GROUP BY c."Nome"
ORDER BY "Media_Recensioni" DESC;
```

Query 3: utenti più attivi o con il numero maggiore di complimenti

```
--Utenti piu' attivi o con il maggior numero di complimenti
SELECT "Nome",
       "Num_recensioni",
       ("Num_cool" + "Num_funny" + "Num_useful") AS "Totale_Complimenti"
FROM "UTENTE"
ORDER BY "Num_recensioni" DESC, "Totale_Complimenti" DESC
FETCH FIRST 10 ROWS ONLY;
```

Query 4: distribuzione temporale dei check-in

```
--Distribuzione temporale dei check-in
SELECT TO_CHAR("Timestamp", 'HH24') AS "Fascia_Oraria",
       COUNT(*) AS "Numero_Checkin"
FROM "CHECKIN"
GROUP BY TO_CHAR("Timestamp", 'HH24')
ORDER BY "Fascia_Oraria";
```

Query 5: categorie di attività con maggiore densità di fotografie

```
--Categorie di attivita' con maggiore densita' di fotografie
SELECT c."Nome" AS "Categoria",
       COUNT(f."ID_Foto") AS "Totale_Foto"
FROM "CATEGORIA" c
JOIN "APPARTIENE_A" aa ON c."ID_Categoria" = aa."ID_Categoria"
LEFT JOIN "FOTO" f ON aa."ID_Attivita" = f."ID_Attivita"
GROUP BY c."Nome"
ORDER BY "Totale_Foto" DESC;
```

Query 6: attività della categoria 'Italian' con più recensioni e foto

```
SELECT
    a."Nome",
    a."Num_recensioni",
    COUNT(f."ID_Foto") AS "Totale_Foto",
    (a."Num_recensioni" + COUNT(f."ID_Foto")) AS "Totale_Combinato"
FROM "ATTIVITA" a
JOIN "APPARTIENE_A" aa ON a."ID_Attivita" = aa."ID_Attivita"
JOIN "CATEGORIA" c ON aa."ID_Categoria" = c."ID_Categoria"
LEFT JOIN "FOTO" f ON a."ID_Attivita" = f."ID_Attivita"
WHERE c."Nome" = 'Italian'
GROUP BY a."ID_Attivita", a."Nome", a."Num_recensioni"
ORDER BY "Totale_Combinato" DESC
FETCH FIRST 10 ROWS ONLY;
```

Query 7: distribuzione percentuale delle recensioni delle attività con attributo 'GoodForKids'

```
SELECT
    r."Stelle",
    COUNT(*) AS "Conteggio_Recensioni",
    ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER (), 2) AS "Percentuale"
FROM "RECENSIONE" r
JOIN "POSSIEDE" p ON r."ID_Attivita" = p."ID_Attivita"
JOIN "ATTRIBUTO" attr ON p."ID_Attributo" = attr."ID_Attributo"
WHERE attr."Nome" = 'GoodForKids'
GROUP BY r."Stelle"
ORDER BY r."Stelle" DESC;
```

5. Oracle APEX

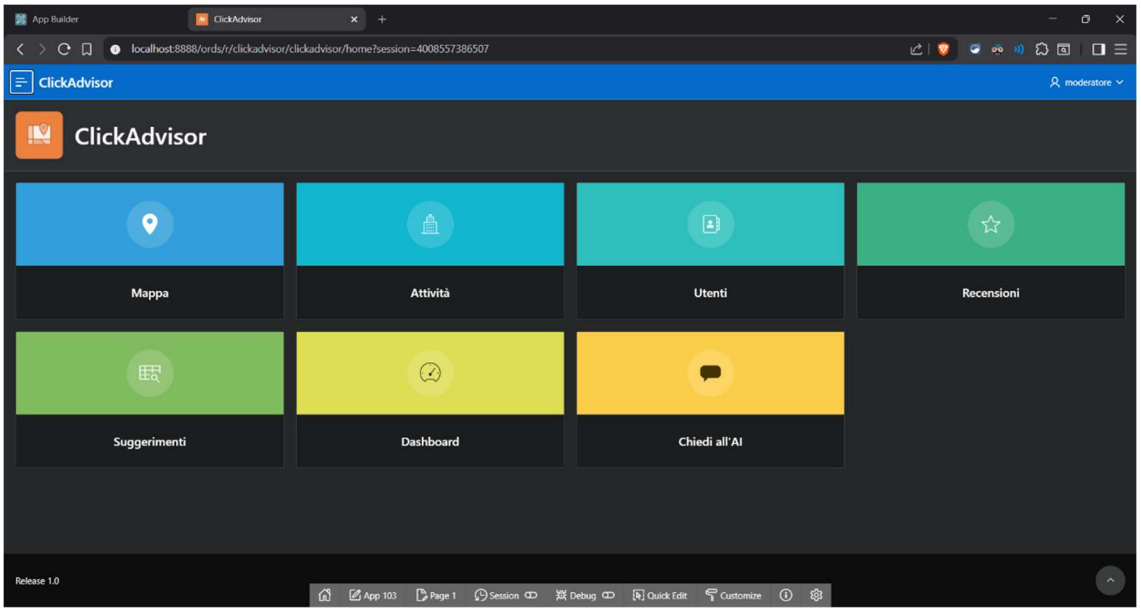
Oracle APEX (Application Express) è una piattaforma di sviluppo low-code che permette di creare applicazioni web aziendali (gestionali, dashboard, report) in modo rapido, utilizzando una piattaforma web-based.

5.1 Pagine realizzate

Di seguito una lista, corredata da screenshot, delle pagine realizzate per la nostra applicazione web.

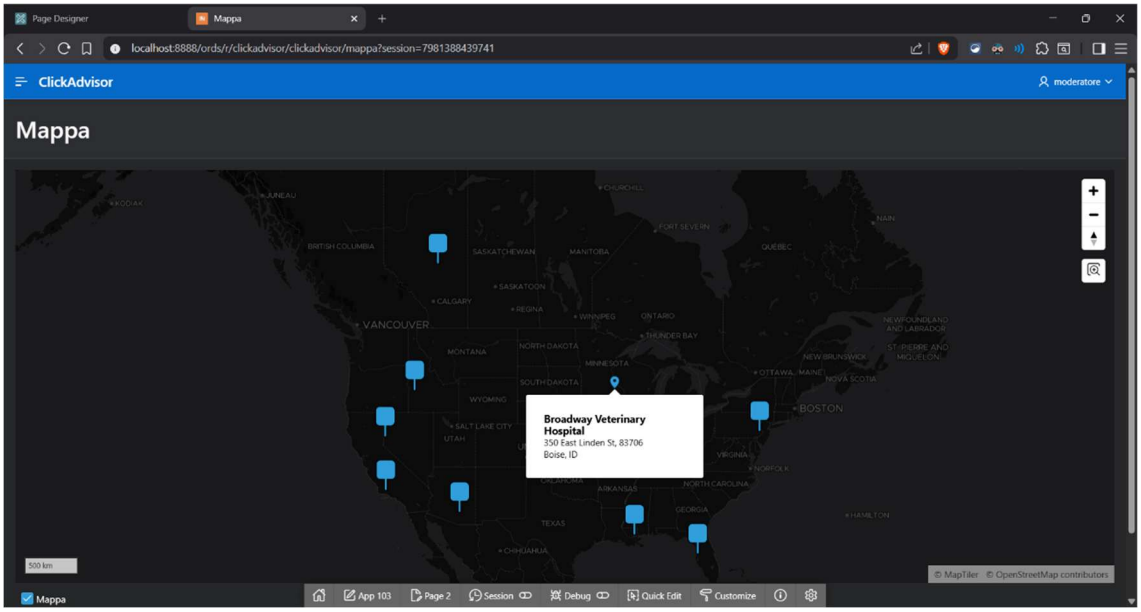
Pagina Home:

La pagina iniziale della nostra applicazione, presenta link che portano in maniera rapida alle altre pagine.



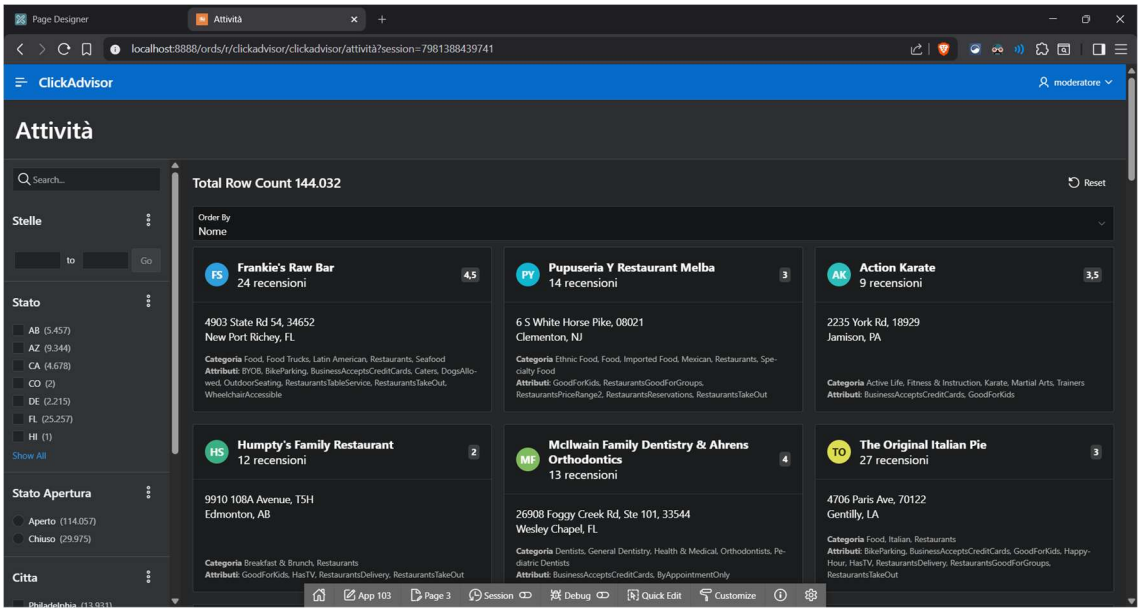
Pagina Mappa:

Mostra una mappa con tutte le attività presenti nel database.



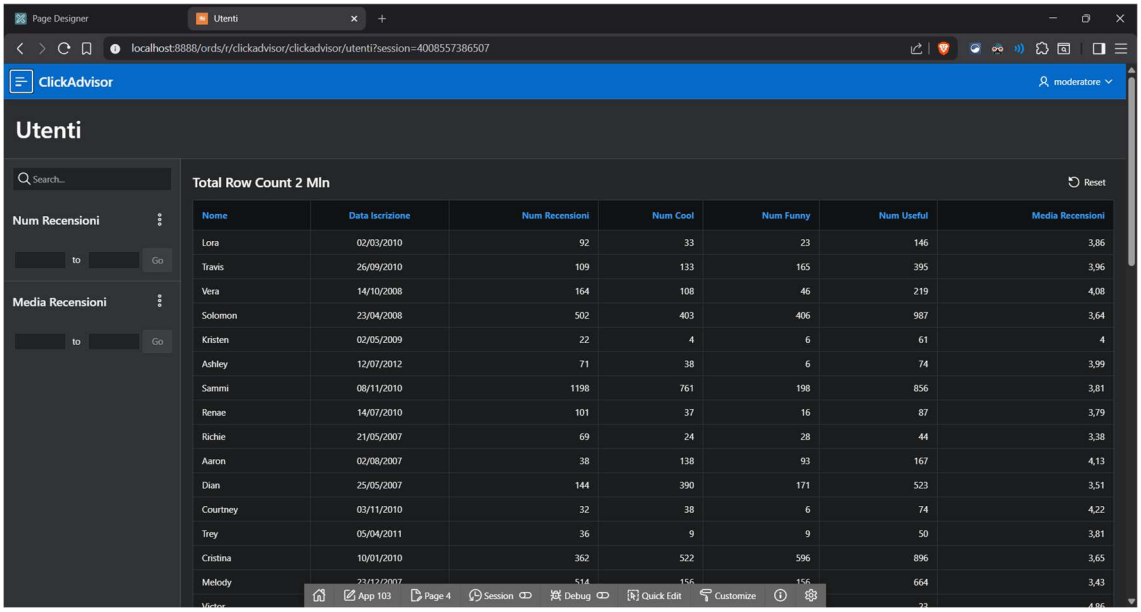
Pagina Attività:

Mostra in dettaglio le attività registrate nel database:



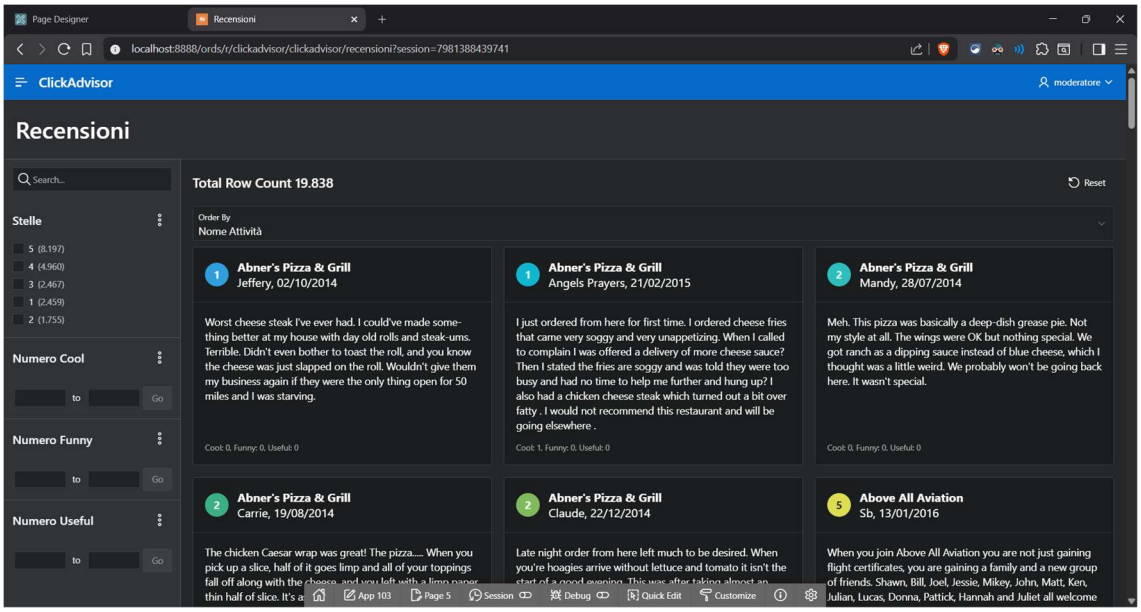
Pagina Utenti:

Mostra tutti gli utenti presenti nel database.



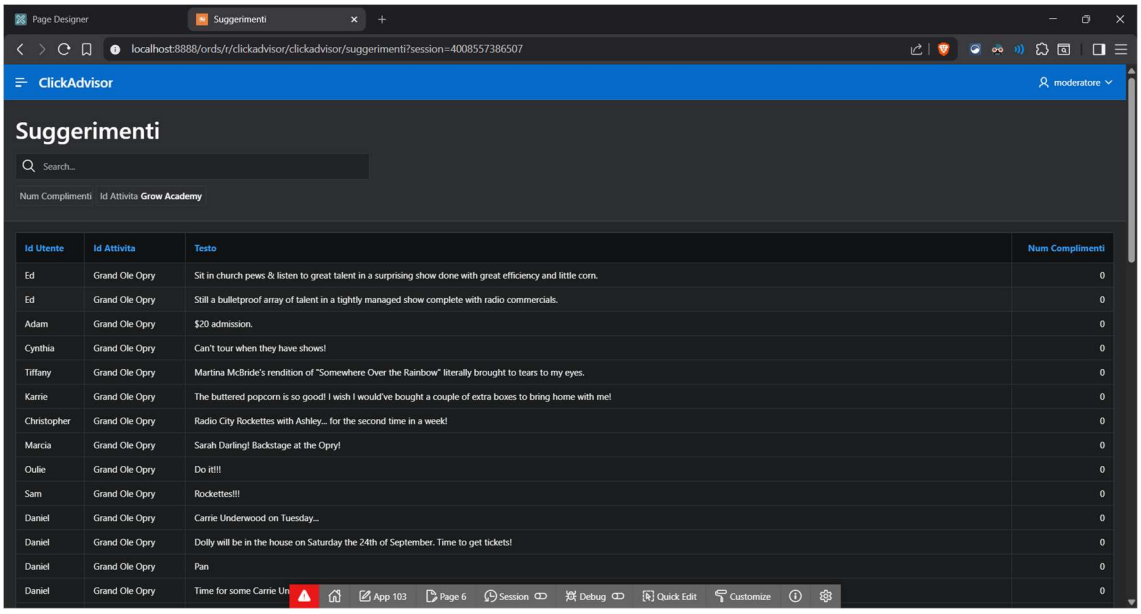
Pagina Recensioni:

Mostra tutte le recensioni presenti nel database.



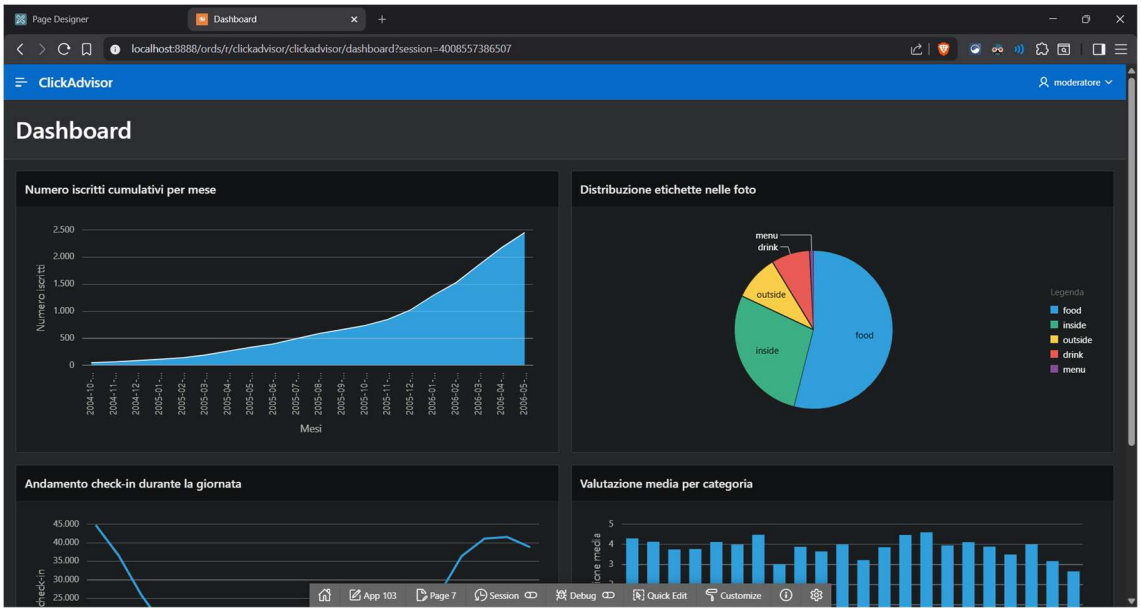
Pagina Suggerimenti:

Mostra una lista dei suggerimenti presenti nel database.



Pagina Dashboard:

Mostra una serie di grafici con statistiche interessanti sui comportamenti degli utenti.



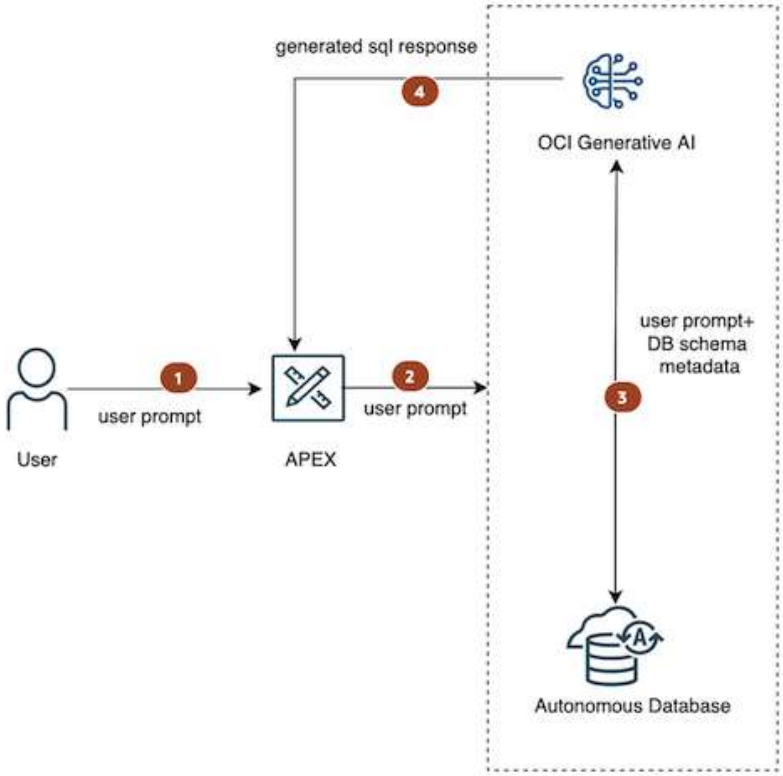
5.2 Query in linguaggio naturale.

Per decenni, l'accesso alle informazioni contenute nei database è stato mediato dalla conoscenza obbligatoria del SQL: un linguaggio potente, ma caratterizzato da una sintassi rigida che agisce spesso come barriera tra il dato e l'utente non tecnico.

Oggi, l'avvento dell'Intelligenza Artificiale Generativa e dei **Large Language Models** (LLM) sta abbattendo questa barriera, introducendo il concetto di **Natural Language Query** (NLQ).

Per rendere possibile l'interazione in linguaggio naturale tra utente e base di dati di ClickAdisor è stato utilizzato lo strumento di Oracle Cloud Infrastructure (OCI) Select AI.

Select AI è un componente di Oracle APEX (package name DBMS_CLOUD_AI) che permette al nostro database di interagire con diversi LLM per generare query a partire dal prompt utente e dallo schema del database. Di seguito è raffigurato lo schema dell'architettura di questo componente:



Una volta che l'user ha inserito un prompt sull'applicazione APEX, Select AI si occuperà di inviare all'LLM di nostra scelta le informazioni sulla struttura del database e l'user prompt, dopodiché trasferirà il risultato nuovamente su APEX per poterlo visualizzare.

Per installare Select AI eseguiremo una serie di comandi volti a dare al package i grant necessari ad interrogare il database; da notare il comando CREATE_PROFILE con cui possiamo selezionare l'LLM che vogliamo utilizzare (in questo caso Cohere, modello supportato direttamente dall'infrastruttura di Oracle Cloud), lo schema su cui deve operare e la temperatura (cioè quanto il modello si deve attenere al prompt utente).

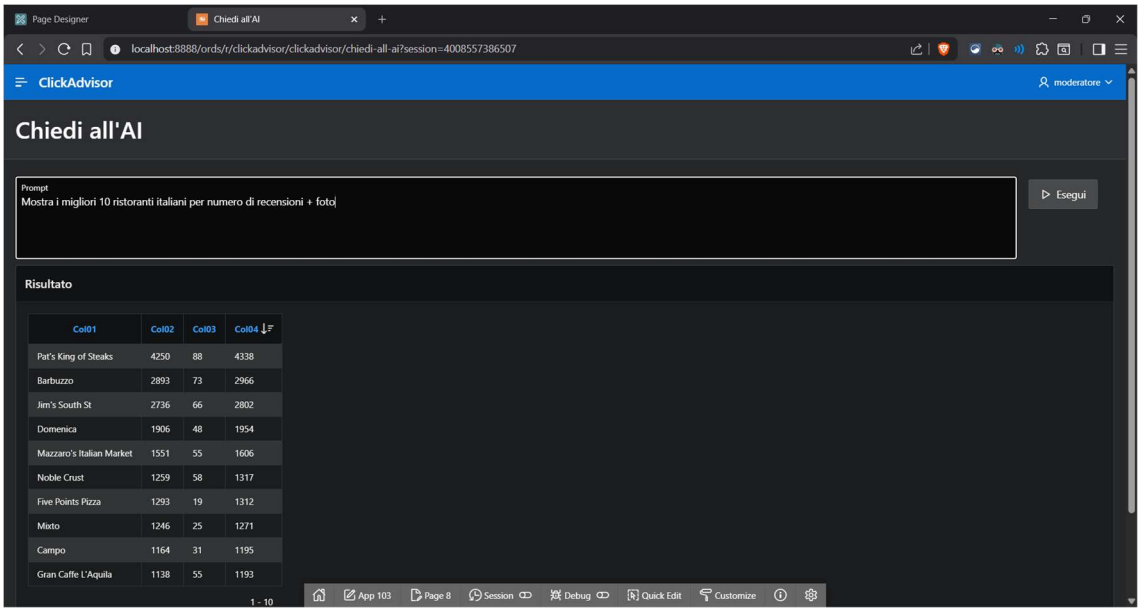
Di seguito i comandi per installare Select AI:

```
--Diamo i diritti di utilizzo di Select AI all'amministratore
GRANT EXECUTE ON DBMS_CLOUD_AI TO AMMINISTRATORE_DB;

--Diamo i permessi di amministratore a Select AI
BEGIN
    DBMS_CLOUD_ADMIN.ENABLE_PRINCIPAL_AUTH(provider => 'OCI',username
=>'AMMINISTRATORE_DB');
END;

--Creiamo un profilo per configurare Select AI sul nostro DB
BEGIN
    DBMS_CLOUD_AI.CREATE_PROFILE(
    'OCI_GENAI',
    '{
        "provider": "oci",
        "credential_name": "OCI$RESOURCE_PRINCIPAL",
        "object_list": [
            {
                "owner": "AMMINISTRATORE_DB"
            }
        ],
        "model": "cohere.command-light",
        "oci_runtimetype": "COHERE",
        "temperature":"0.4"
      }');
END;
EXEC DBMS_CLOUD_AI.set_profile('OCI_GENAI');
```

Una volta installato sarà possibile inviare prompt usando questo comando:

```
--Esempio di prompt
SELECT DBMS_CLOUD_AI.GENERATE(
    prompt       => 'Mostra tutte le attivita aperte il lunedi alle 10',
    profile_name => 'OCI_GENAI'
)
FROM dual;
```

Di seguito vediamo la pagina realizzata su APEX per effettuare query in linguaggio naturale implementando Select AI:



La pagina presenta un text field per l'inserimento del prompt, un pulsante per confermare l'invio della query e un classic report per visualizzare i dati prodotti tramite DBMS_CLOUD_AI.